

Like and Comment Functionality Implementation Summary

Overview

This implementation adds like and comment functionality to community posts (exercise submissions) in The Writer's Corner application.

Technology Stack

- **Frontend:** Next.js 14 (App Router), React 18, TypeScript
- **Backend:** Next.js API Routes
- **Database:** PostgreSQL with Prisma ORM
- **Authentication:** NextAuth.js (JWT-based sessions)
- **UI:** Tailwind CSS, Radix UI, shadcn/ui components
- **Notifications:** react-hot-toast

Database Schema Changes

New Models

1. SubmissionLike

Tracks likes on exercise submissions (community posts).

- `id` : Unique identifier (cuid)
- `userId` : Reference to the user who liked
- `submissionId` : Reference to the exercise submission
- `createdAt` : Timestamp when like was created
- **Unique constraint:** [`userId`, `submissionId`] (prevents duplicate likes)

2. SubmissionComment

Stores comments on exercise submissions.

- `id` : Unique identifier (cuid)
- `content` : Comment text (Text field)
- `userId` : Reference to the comment author
- `submissionId` : Reference to the exercise submission
- `createdAt` : Timestamp when comment was created
- `updatedAt` : Timestamp when comment was last updated

3. PostLike & PostComment

Similar models for the CommunityPost entity (prepared for future use).

Updated Models

- **User:** Added relations for `submissionLikes`, `submissionComments`, `postLikes`, and `postComments`
- **ExerciseSubmission:** Added relations for `likes` and `comments`
- **CommunityPost:** Added relations for `likes` and `comments`

API Endpoints

Like Endpoints

POST /api/community/posts/[id]/like

Toggles like status for a post (like/unlike).

- **Authentication:** Required
- **Request:** None (post ID in URL)
- **Response:**

```
json
{
  "liked": true,
  "likeCount": 5
}
```

GET /api/community/posts/[id]/like

Gets like status and count for a post.

- **Authentication:** Required
- **Response:**

```
json
{
  "liked": false,
  "likeCount": 5
}
```

Comment Endpoints

GET /api/community/posts/[id]/comments

Retrieves all comments for a post.

- **Authentication:** Required
- **Response:**

```
json
{
  "comments": [
    {
      "id": "...",
      "content": "Great work!",
      "createdAt": "2025-11-01T...",
      "user": {
        "id": "...",
        "firstName": "John",
        "lastName": "Doe",
        "name": "John Doe"
      }
    }
  ]
}
```

POST /api/community/posts/[id]/comments

Creates a new comment on a post.

- **Authentication:** Required

- Request:

```
json
{
  "content": "This is a great post!"
}
```

- Response:

```
json
{
  "comment": {
    "id": "...",
    "content": "This is a great post!",
    "createdAt": "...",
    "user": { ... }
  }
}
```

DELETE /api/community/posts/[id]/comments/[commentId]

Deletes a comment (author only).

- **Authentication:** Required
- **Authorization:** Only the comment author can delete
- **Response:**

```
json
{
  "message": "Comment deleted successfully"
}
```

Updated Endpoint**GET /api/community/posts**

Enhanced to include like and comment counts for each post.

- Added `likeCount`, `commentCount`, and `isLikedByUser` fields to response

UI Components**New Component: PostComments**

File: `app/components/community/post-comments.tsx`

A comprehensive comment section component that:

- Displays all comments for a post with author information
- Shows relative timestamps (e.g., "2h ago", "Just now")
- Provides a textarea for writing new comments
- Includes delete functionality for own comments
- Features loading states and empty states
- Integrates with react-hot-toast for notifications

Props:

- `postId` : The ID of the post to show comments for
- `isOpen` : Boolean to control visibility
- `onClose` : Callback to close the comment section

Updated Component: CommunityOverview

File: `app/components/community/community-overview.tsx`

Enhanced with:

- **Like button:**
 - Shows like count or “Like” text
 - Filled heart icon when liked
 - Color changes based on like status (red when liked)
 - Toggles like on click
- **Comment button:**
 - Shows comment count or “Comment” text
 - Toggles comment section visibility
 - Color changes when comment section is open
- **PostComments integration:**
 - Renders below each post when toggled
 - Only one comment section open at a time
- **Real-time updates:**
 - Like counts update immediately
 - Comment counts increment when comments are added
 - UI reflects current user’s like status

Features Implemented

Like Functionality

- [x] Users can like posts
- [x] Users can unlike posts (toggle)
- [x] Like count displayed on each post
- [x] Visual feedback (filled heart) when liked
- [x] Real-time like count updates
- [x] Prevents duplicate likes (database constraint)

Comment Functionality

- [x] Users can comment on posts
- [x] Comments display with author information
- [x] Comments show relative timestamps
- [x] Users can delete their own comments
- [x] Comment count displayed on each post
- [x] Real-time comment list updates
- [x] Toast notifications for actions

Authentication & Authorization

- [x] All endpoints require authentication
- [x] Users can only delete their own comments

- [x] User association tracked for all likes and comments

Security Considerations

1. **Authentication:** All API endpoints verify user authentication via NextAuth session
2. **Authorization:** Comment deletion restricted to comment authors
3. **Input Validation:** Comment content validated for non-empty strings
4. **Database Constraints:** Unique constraints prevent duplicate likes
5. **Cascade Deletes:** Comments and likes deleted when user or post is deleted

Future Enhancements

Potential improvements for future iterations:

- Edit functionality for comments
- Nested replies (threaded comments)
- Like/comment notifications
- Pagination for comments
- Real-time updates with WebSockets
- Comment reactions (emoji reactions)
- Mention system (@user)
- Rich text formatting for comments
- Comment sorting options
- Flag/report inappropriate comments

Migration Notes

When deploying to production:

1. Run Prisma migration: `npx prisma migrate dev --name add_likes_comments`
2. Generate Prisma client: `npx prisma generate`
3. Restart the application to load new schema
4. Test all endpoints with authenticated users

Testing Checklist

Before deployment, verify:

- [] Can create likes on posts
- [] Can remove likes from posts
- [] Like count updates correctly
- [] Can create comments on posts
- [] Comments display with correct author info
- [] Can delete own comments
- [] Cannot delete others' comments
- [] Comment count updates correctly
- [] Unauthenticated users cannot like/comment
- [] UI updates in real-time
- [] Toast notifications work correctly
- [] No console errors in browser

Files Changed/Added

Modified Files

1. `app/prisma/schema.prisma` - Database schema with new models
2. `app/app/api/community/posts/route.ts` - Enhanced with like/comment data
3. `app/components/community/community-overview.tsx` - UI for likes and comments

New Files

1. `app/app/api/community/posts/[id]/like/route.ts` - Like toggle endpoint
2. `app/app/api/community/posts/[id]/comments/route.ts` - Comment CRUD endpoints
3. `app/app/api/community/posts/[id]/comments/[commentId]/route.ts` - Comment deletion
4. `app/components/community/post-comments.tsx` - Comment UI component

Dependencies

No new dependencies required. All functionality uses existing packages:

- `@prisma/client` - Database operations
- `next-auth` - Authentication
- `react-hot-toast` - Notifications (already in use)
- `lucide-react` - Icons (already in use)